



Model-Guided Fuzzing of Distributed Systems

EGE BERKAY GULCAN, Delft University of Technology, Netherlands

BURCU KULAHCIOGLU OZKAN, Delft University of Technology, Netherlands

RUPAK MAJUMDAR, MPI-SWS, Germany

SRINIDHI NAGENDRA, IRIF - CNRS - Université Paris Cité, France and Chennai Mathematical Institute, India

We present a **coverage-guided testing algorithm** for distributed systems implementations. Our main innovation is the use of an abstract formal model of the system that is used to define coverage. Such abstract models are frequently developed in the early phases of protocol design and verification but are infrequently used at testing time. We show that guiding random test generation using model coverage can be effective in covering interesting points in the implementation state space. We have implemented a fuzzer for distributed system implementations and abstract models written in TLA+. Our algorithm achieves better coverage over purely random exploration as well as random exploration guided by different notions of scheduler coverage and mutation. In particular, we show consistently higher coverage on implementations of distributed consensus protocols such as Two-Phase Commit and the Raft implementations in Etcd-raft and RedisRaft and detect bugs faster. Moreover, we discovered 12 previously unknown bugs in their implementations, four of which could only be detected by model-guided fuzzing.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**; • **Theory of computation** → **Distributed computing models**.

Additional Key Words and Phrases: Random testing, Fuzzing, Formal models, Distributed systems

ACM Reference Format:

Ege Berkay Gulcan, Burcu Kulahcioglu Ozkan, Rupak Majumdar, and Srinidhi Nagendra. 2025. Model-Guided Fuzzing of Distributed Systems. *Proc. ACM Program. Lang.* 9, OOPSLA2, Article 282 (October 2025), 28 pages. <https://doi.org/10.1145/3763060>

1 Introduction

Large-scale distributed systems form the core infrastructure for many software applications. It is well-known that designing such systems is difficult and error-prone due to the interaction between concurrency and faults, and subtle bugs often show up in production. Thus, designing testing techniques that cover diverse and interesting program behaviors to find subtle bugs has been an important research challenge.

Coverage-guided fuzzing, which guides test generation toward more coverage, has been effective in exploring diverse executions, mainly in the sequential setting, using structural coverage criteria as a feedback mechanism [32, 75]. However, adopting coverage-guided fuzzing for testing distributed system implementations is nontrivial since there is no common notion of *coverage* for distributed system executions. Unfortunately, structural code coverage criteria such as line coverage can ignore the orderings of message interactions in a system, thus missing interesting schedules. On the other

Authors' Contact Information: Ege Berkay Gulcan, Delft University of Technology, Delft, Netherlands, E.B.Gulcan@tudelft.nl; Burcu Kulahcioglu Ozkan, Delft University of Technology, Delft, Netherlands, b.ozkan@tudelft.nl; Rupak Majumdar, MPI-SWS, Kaiserslautern, Germany, rupak@mpi-sws.org; Srinidhi Nagendra, IRIF - CNRS - Université Paris Cité, Paris, France and Chennai Mathematical Institute, Chennai, India, nagendra@irif.fr.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2025 Copyright held by the owner/author(s).

ACM 2475-1421/2025/10-ART282

<https://doi.org/10.1145/3763060>

hand, more detailed criteria, such as traces of messages, may provide too many coverage goals and thus consider each random trace a new behavior, giving up the advantages of coverage-guided exploration.

In this paper, we propose to use *state coverage* in an *abstract formal model* of the system as a coverage criterion and present *model-guided fuzzing* of distributed systems. Abstract formal models are often developed in the design phase of distributed systems to model and formally analyze the underlying protocols [11, 16, 17, 19, 39, 53]. We show that these artifacts are also beneficial in the continuous testing infrastructure of the implementations themselves. Our experiments show that a formal model can serve as a good “guide” for a random testing engine—this is because the formal model often captures the important scenarios of the protocol, and coverage of states in the model correlates well with coverage of interesting behaviors in the implementation.

Our motivation for using abstract model guidance to generate test executions shares common insights with semantic fuzzing [55] and grammar-based fuzzing approaches [30, 40] used for testing sequential programs. Semantic fuzzing aims to cover interesting program executions processing program inputs rather than spending exploration budget for exercising uninteresting, syntactic input parsing logic. While a naive fuzzer is likely to generate inputs that cannot pass the input validation and parsing stage, semantic fuzzing generates test inputs that can go deep into the execution. Similarly, a naive event scheduler for distributed systems is likely to produce tests that spend execution budget in exercising uninteresting, network setup stages. For example, it can explore many different orderings of vote messages during the cluster’s leader election phase, barely electing a leader after a prolonged execution. Our approach aims to direct testing toward interesting system behaviors, e.g., processing of user requests once a leader is elected. Similar to grammar-based fuzzing that uses formal specification of the test input to guide test generation, we use an abstract formal model of distributed systems to guide the generation of semantically interesting temporal event schedules.

At a high level, the abstract models recognize semantically interesting behavior; the use of abstract states is a way to provide coverage criteria that capture program semantics. Of course, the use of abstract models is not a panacea: the abstraction may not cover certain implementation details where bugs may lurk. However, lack of structural coverage after model-guided exploration can indicate where additional testing effort should focus, as well as point out aspects of implementation behavior that are not covered by the model.

We have implemented our algorithm for testing distributed systems implementations using TLA+ models [39] of protocols. In a nutshell, our testing algorithm proceeds as follows. We start by exploring random schedules of messages, but feed the same sequence of messages to the TLA+ model. We use the TLC model checker [72] to generate the set of reachable model states corresponding to the explored schedule. We mark a schedule as “interesting” if it covers a new state of the abstract model. We perform, as in coverage-guided fuzzing, a *mutation* of an interesting schedule by swapping the receipt order of two randomly chosen messages or changing the processes to crash. Applying a mutation to an event schedule gives a new schedule to explore that is similar to the original schedule but likely to exercise new system behavior.

We applied our algorithm to test an implementation of the Two-Phase Commit protocol and two industrial implementations of Raft in Etcd-raft and RedisRaft. Our evaluation shows that model-guided fuzzing leads to higher coverage and can detect bugs faster than pure (unguided) random testing, structural code-coverage guided fuzzing, and trace-based coverage-guided fuzzing. Besides reproducing known bugs, we discovered 12 previously unknown bugs in the implementations of Etcd-raft and RedisRaft. More importantly, four of the new bugs could only be detected by our model-guided fuzzing approach.

Overall, we make the following contributions:

- (1) We propose using abstract models of the systems to guide the testing of system implementations and present ModelFuzz, a model-guided fuzzing approach for distributed systems.
- (2) We implement ModelFuzz for testing the two production implementations of the Two-Phase Commit and Raft protocols and evaluate it compared to the existing approaches.
- (3) We discovered 12 unknown bugs in total in the implementations of Etcd-raft and RedisRaft.

2 Overview

In this section, we motivate and overview the use of state coverage in an abstract system model to guide test generation on an example distributed system. First, we describe the example system with a concurrency bug in its implementation. Then, we motivate coverage-guided testing to detect such bugs more effectively. Finally, we show that model-guided coverage provides more useful information in guiding test generation than other coverage notions.

An example distributed system. Figure 1a presents an example system implemented in a Coyote-like actor programming framework [1, 17]. The system consists of three processes AppMaster, Worker, and Terminator. Each runs in a separate process and communicates with each other by exchanging asynchronous messages. AppMaster receives client requests, coordinates their execution by a Worker process, and manages the termination of the worker using a Terminator process. It accepts the registration messages from Worker and Terminator and registers them (lines 5-11). Upon receipt of a client request Request(*r*), it checks whether the cluster is ready by checking the registrations of the worker and terminator. If it is ready, it sends Execute(*r*) to Worker to handle the request and sends Terminate(*w*) to Terminator (lines 13-17). The Worker and Terminator register to the AppMaster upon their initialization (lines 26, 39). The Worker handles Execute(*r*) by processing it (line 30) and Flush by cleaning up its buffers (line 34). The Terminator handles Terminate by sending Flush to the worker (line 42).

The above implementation has a message race bug [42], which occurs in a particular delivery ordering of the asynchronous messages. The worker code mistakenly omits to check if the buffer is null before processing a task (line 31). This causes the worker to access a null pointer while processing the client request if Flush is delivered to it before Execute(*r*). Figures 1b and 1c illustrate a correct and an incorrect execution of the implementation.

Although the bug seems simple, it is hard to discover such bugs using naive random testing. Manifesting the bug requires reaching a system state that allows the system to produce *interesting* executions. In our example, the processes must register themselves to the AppMaster before the system serves client requests. Thus, pure random testing can be ineffective at exposing distributed system bugs since the generated test cases can get stuck in uninteresting parts of the execution space or explore redundant executions.

Coverage-guided testing. Coverage-guided techniques guide the generation of test executions toward unexplored system behaviors to increase test coverage and search for bugs more efficiently. These methods track the coverage of each test execution and use this information while generating new test cases. However, structural test coverage metrics, such as line or branch coverage, are ineffective for assessing the coverage of distributed system executions since they do not capture the concurrency behavior of the distributed systems. For example, both correct and buggy executions in Figures 1b and 1c hit exactly the same lines and branches of the program in Figure 1a but with different processing orders of the messages.

Model coverage. Instead, we propose to use an abstract, formal, model of the system (e.g., its TLA+ model [39]) to assess test coverage and guide the test generation.

```

1  class AppMaster: StateMachine {
2      StateMachine worker;
3      StateMachine terminator;
4
5      def onRegister(Register r) = {
6          StateMachine m = r.stateMachine;
7          if (m instanceof Worker)
8              worker = m;
9          else if (m instanceof Terminator)
10             terminator = m;
11     }
12
13     def onRequest(Request r) = {
14         if (isReady()) {
15             worker.send(Execute(r));
16             terminator.send(Terminate(worker));
17         }
18     }
19     def isReady(): Boolean = {
20         return worker != null
21             && terminator != null; }
22 }
23
24 class Worker: StateMachine {
25     Buffer buffer;
26
27     def onInit(StateMachine a) = {
28         initBuffer(buffer);
29         a.send(Register(this)); }
30
31     def onExecute(Request r) = {
32         // if (buffer != null)
33         runTask(buffer, r); }
34
35     def onFlush() = { buffer = null; }
36 }
37
38 class Terminator: StateMachine {
39     def onInit(StateMachine a) = {
40         a.send(Register(this)); }
41
42     def onTerminate(Worker w) = {
43         w.send(Flush); }
44 }

```

(a) An example system with three processes AppMaster, Worker, and Terminator

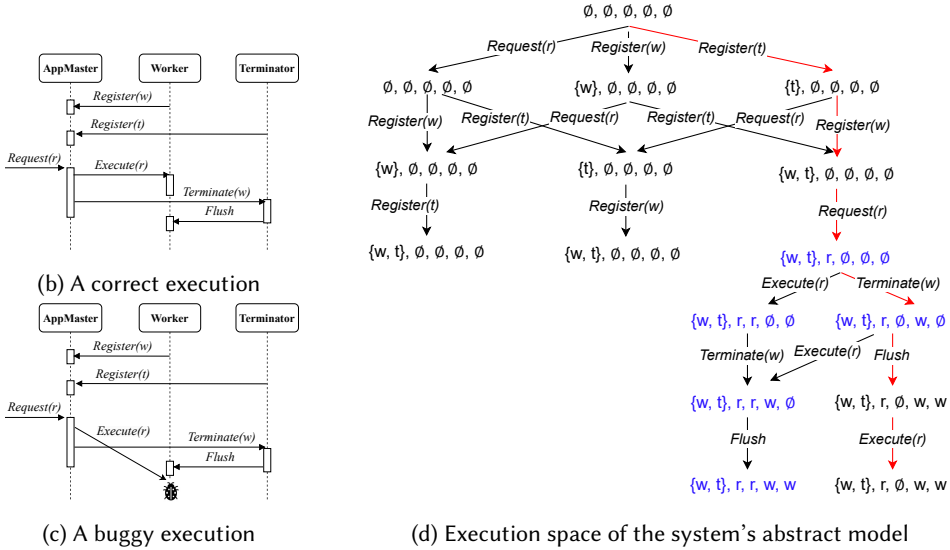


Fig. 1. An example distributed system, its two possible executions, and the execution space of the system's abstract model.

Figure 1d shows the execution space of the abstract system model. We consider the TLA⁺ model of the system¹, which describes the system's behavior as a transition system specified by a set of states and a set of transitions between them. The figure illustrates the possible executions of

¹<https://github.com/burcuku/tlc-controlled-with-benchmarks>

the system, including the system states and transitions, where each transition corresponds to the delivery of a particular message. A state of the TLA^+ model of the system is defined by (i) the state of the AppMaster, which keeps the set of registered processes *registered* and the set of client requests to process *requests*, (ii) the state of the Worker, which keeps the set of completed tasks, *completed*, and (iii) the state of the Terminator, which keeps the set of tasks to terminate, *toTerminate* and the set of terminated tasks *terminated*. For simplicity, we denote the state of the system as a tuple $\langle registered, requests, completed, toTerminate, terminated \rangle$. The possible actions in the system are the delivery of the messages *Request(r)*, *Register(w)*, *Register(t)*, *Execute(r)*, *Terminate(w)*, and *Flush*, which match the delivery of the messages *Request(r)*, *Register(w)*, *Register(t)*, *Execute(r)*, *Terminate(w)*, and *Flush* in the system's implementation, respectively. As given in Figure 1d, the execution starts in the initial state $\langle \emptyset, \emptyset, \emptyset, \emptyset, \emptyset \rangle$ and updates the system state based on the actions taken. Our approach assesses the coverage of a set of executions by measuring their *state coverage* in the abstract formal model.

We could have alternatively considered *trace-based* coverage based on the Mazurkiewicz traces arising out of an execution. Mazurkiewicz traces associate two sequences of messages with the same Mazurkiewicz trace if they only reorder *independent* messages, whose reordering does not affect the system's behavior. Our state-based notion is coarser than traces while keeping the essential information about the system's behavior. For example, although the messages *Register(w)* and *Register(t)* are dependent, their relative ordering does not affect the system state. On the other hand, the ordering of *Request(r)* affects the reached system state; the system handles the request only if it is delivered after the two registration messages. The given system has 10 possible message orderings with 8 Mazurkiewicz traces (capturing the commutativity of the *Execute* and *Terminate* messages). However, the set of all possible system states can be covered by running fewer executions, e.g., only 2 executions for this example.

Model-guided testing. Now, we show that guiding the generation of test executions using the model state coverage directs the exploration toward interesting behaviors more effectively than with trace-based coverage. This is because model-state based coverage captures the covered set of program behavior more succinctly than trace-based coverage, which labels each reordering of dependent events as a new coverage class regardless of the system behavior.

Consider an example where the following set of executions has been explored:

- E1 *Request(r)*, *Register(w)*, *Register(t)*
- E2 *Register(w)*, *Request(r)*, *Register(t)*
- E3 *Request(r)*, *Register(t)*, *Register(w)*
- E4 *Register(t)*, *Request(r)*, *Register(w)*
- E5 *Register(w)*, *Register(t)*, *Request(r)*, *Execute(r)*, *Terminate(w)*, *Flush*
- E6 *Register(t)*, *Register(w)*, *Request(r)*, *Terminate(w)*, *Execute(r)*, *Flush*

Coverage-guided testing selects the executions that hit previously unseen coverage classes, and it generates new test executions by mutating them. Trace-based coverage would label all these executions as *interesting* since each belongs to a different coverage class (i.e., they deliver *Request(r)*, *Register(w)*, and *Register(t)* to the same process in a different order). Generating new tests around all of these executions is likely to result in many redundant runs since many of them (E1-E4) already demonstrate the same system behavior.

In contrast, state-based coverage identifies the coverage of new states in the executions of E5 and E6, which hit some new system states (marked blue in Figure 1d) that are not observed in E1-E4. Therefore, state-based coverage-guided testing generates new test cases only around these executions. A mutation that swaps the order of *Execute(r)* and *Flush* in the execution (marked by the red arrows in the figure) triggers the concurrency bug in the implementation:

E7 Register(t), Register(w), Request(r), Terminate(w), Flush, Execute(r)

In summary, while naive random testing tends to spend the exploration budget exercising the reorderings of the first messages in Figure 1d and trace-based coverage guidance promotes exploration around all unique traces, our model-based coverage-guided testing directs the execution to the unexplored, interesting executions.

3 Coverage-Guided Fuzzing with Abstract Models

We use *coverage-guided fuzzing* for exploring distributed system executions and guiding the test generation using abstract model coverage. Our method is *complementary* to the traditional fuzzing methods (e.g., AFL [74]), which explore the space of *program inputs*, as ModelFuzz explores the space of *event schedules* in distributed systems. In this section, we define executions and explain how we adopt the coverage-guided fuzzing approach for the model-guided exploration of distributed system executions.

3.1 Executions of Distributed Systems

A distributed system S consists of a set of processes that concurrently operate on their local states and communicate by exchanging asynchronous *messages*. The processes are equipped with message buffers that keep the messages sent to that process, and they process the messages in their buffers serially. Upon handling a message, a process can update its local state and/or send new messages to the processes.²

Let Procs be the set of processes and Msgs be the set of all messages exchanged in the system. We represent the delivery of a message $\text{msg} \in \text{Msgs}$ to its receiver process by an event $e = \langle \text{recv}, \text{send}, \text{msg} \rangle$ for $\text{recv}, \text{send} \in \text{Procs}$, where $\text{recv}(e)$ is the receiver of the message, $\text{send}(e)$ the sender, and $\text{msg}(e)$ is the message. Let Σ be the set of all message delivery events. A *state* s of the system is $s : \langle \text{Buffer} : \text{Procs} \times \text{Procs} \mapsto [\text{Msgs}] \rangle$ where Buffer is a map from a sender-receiver pair of processes to the list of messages in their message buffers. A transition in the system picks a buffer buff and executes the first message msg in it, i.e., running an event $e = \langle \text{recv}, \text{send}, \text{msg} \rangle$. Executing the message can lead to the creation of messages m_i sent to recv , i.e., send may send new messages to some processes upon processing m . The new state s' is obtained by removing m from $s(\text{buff})$ and appending m_i to $s(\text{Buffer}(\text{send}, \text{recv}))$ for each i , and we write $s \xrightarrow{e} s'$. An *execution* of the distributed system is a sequence $s_0 \xrightarrow{e_0} s_1 \xrightarrow{e_1} \dots \xrightarrow{e_n} s_{n+1}$ of states s_i and events e_i . In addition to delivering messages, we introduce two other types of events $\langle \text{proc}, \text{crash} \rangle$, and $\langle \text{proc}, \text{restart} \rangle$, which respectively correspond to crash or restart events of a process proc . We call the sequence $\langle e_0, \dots, e_n \rangle$ an *event schedule*.

3.2 Coverage-Guided Fuzzing of Distributed Systems

Algorithm 1 shows the coverage-guided fuzzing algorithm for generating test inputs [8, 75], which we extended for generating test schedules using coverage-guided fuzzing with abstract models.

The coverage-guided fuzzing algorithm takes the program under test (S) and a set of initial test cases (T_0) as inputs. It maintains the set of test cases to explore (T) and the total coverage (*totalCoverage*) of the executed test cases. After each test execution (line 5), the algorithm checks if the execution covers new system behavior (line 6). If the test case covers new behavior, it assigns an energy value to the test case proportional to the new coverage it achieves (line 7) to explore more around the executions that cover more new behavior. The algorithm generates new test cases by mutating such executions, adds them to the set of executions to explore (lines 8-9), and updates

²We consider FIFO message queues that preserve the order of messages between the same sender-receiver pairs, as in distributed system frameworks such as P [19], Coyote [17], or Akka [45].

Input: A distributed system S

Input: An abstract model of the system M

Input: A map map from system events to model actions

Input: An initial corpus of test cases T_0

Output: The set of explored test cases T

Output: The set of covered abstract states totalCoverage

```

1 proc coverageGuidedFuzzer( $S, M, T_0$ )
2    $T \leftarrow T_0$  ;  $\text{totalCoverage} \leftarrow \emptyset$ 
3   while test budget not exceeded do
4     for  $t \in T$  do
5        $\text{coverage} \leftarrow \text{executeAndGetCoverage}(S, M, t)$ 
6       if  $\text{coverage} \cap \text{totalCoverage} \neq \emptyset$  then
7          $p \leftarrow \text{assignEnergy}(\text{coverage})$ 
8         for 1 to  $p$  do
9            $T' = \text{mutate}(t)$  ;  $T \leftarrow T \cup T'$ 
10         $\text{totalCoverage} \leftarrow \text{totalCoverage} \cup \text{coverage}$ 
11  return  $T, \text{totalCoverage}$ 

12 proc executeAndGetCoverage( $S, M, t$ )
13   $\text{events} \leftarrow \text{executeConcreteSystem}(S, t)$ 
14   $\text{actions} \leftarrow \text{map}(\text{events})$ 
15   $\text{states} \leftarrow \text{executeAbstractModel}(M, \text{actions})$ 
16  return  $\text{states}$ 

```

Algorithm 1: Coverage-guided fuzzing using abstract models. The statements that differ from traditional fuzzers are highlighted in blue.

the total coverage with the newly covered behavior (line 10). It terminates when a test budget (e.g., specified by a test duration or number of test cases) returns the explored set of test cases together with the test coverage.

Our approach adopts coverage-guided fuzzing for the exploration of distributed system executions by (i) generating **event schedules** rather than program inputs, (ii) defining **mutations on the event schedules** to obtain a new schedule, and (iii) assessing the test coverage based on the coverage in the abstract system model rather than using traditional code coverage metrics. Along with the implementation of the system under test S , our algorithm takes an abstract model M of the system as an additional input, which is used to assess coverage. After running a test schedule on the concrete system implementation S , it collects the sequence of executed events (line 13) and maps that sequence of concrete system events into the sequence of abstract model actions (line 14). The event mapper method map is provided by the developer, which simply maps an event in the system implementation (e.g., delivery of a message to a process) to an action in the abstract system model. After mapping the system events to model actions, the algorithm runs these actions on the systems' abstract model (line 15). It collects the set of abstract states covered in the system model and returns that as the coverage information for the test schedule (line 16).

Event schedules as test cases. A test case corresponds to a schedule of distributed system events $\langle e_0, \dots, e_n \rangle$, where e_i is either (i) the delivery of a message or (ii) crashing a process or (iii) restarting it. We collect the set of initial test schedules (T_0) by randomly scheduling the events in the executions, where we select the next event uniformly at random among the set of all enabled events.

The space of mutations. Mutations are used to extend the search around interesting schedules that cover new coverage goals. They introduce small changes to an event schedule to produce new executions that are similar to the original schedule but likely to exercise new behavior. A difficulty in mutating event schedules is obtaining a feasible schedule after mutating the event order. An offline shuffle of a sequence is likely to produce an infeasible schedule, e.g., the new schedule may not contain all the events that appear in the new execution.

We represent event schedules by referring to the events using the processes that run them rather than directly referring to the delivered messages. We represent a test case as a sequence of abstract events $s = \langle \text{buff}_0 : a_0 \rangle \dots \langle \text{buff}_n : a_n \rangle$ with $a_0, \dots, a_n \in \{\text{deliver}, \text{crash}, \text{restart}\}$. Event $\langle \text{buff}_i : \text{deliver} \rangle$ delivers the first message in buff_i to its recipient process, $\langle \text{buff}_i : \text{crash} \rangle$ crashes the recipient process of buff_i , and $\langle \text{buff}_i : \text{restart} \rangle$ restarts it. The test case does not explicitly refer to the messages delivered by the deliver action (i.e., with certain sender and content) but indirectly refers to them using the message buffers whose messages we deliver. Moreover, we parameterize deliver with the number of messages to deliver, e.g., $\langle \text{buff}_i : \text{deliver}(n) \rangle$ corresponds to delivering n messages from buff_i to the recipient process. The indirect representation of the messages helps design mutations that result in feasible event schedules while modifying the order of events. We use the following mutations to generate new schedules:

- **SwapBuffers** randomly selects two schedule indices i, j in s , and swaps the scheduling order of buff_i and buff_j ,
- **SwapCrashProcesses** randomly selects two schedule indices i, j where the recipient of the buff_i is crashed at step i and buff_j is crashed at step j , and swaps the positions of the crash events (for schedules with a single crashing process, it changes the process to crash),
- **SwapMaxMessages** randomly selects two schedule indices i, j with message delivery events and swaps q_i and q_j , i.e., the number of messages to deliver at these positions.

3.3 Notions of Coverage

The main challenge in designing a coverage-guided testing method for exploring distributed system executions is to **decide whether an execution covers interesting behaviors** to guide the exploration around that execution. Unfortunately, existing code coverage metrics, which are designed for measuring the coverage of sequential programs, or Mazurkiewicz traces, which provide a syntactic definition of equivalent behaviors in concurrent programs, are unsuitable for measuring the coverage of distributed system executions.

Code coverage metrics. The traditional coverage-guided testing algorithms, such as AFL++, use code coverage metrics to check whether an execution covers new behaviors. If the execution of a test input exercises new lines or branches of the program under test, it marks that execution as interesting and generates more test inputs similar to that input. However, the existing code coverage notions used for testing sequential programs are not suitable for measuring the coverage of behaviors of distributed system executions. The same lines or branches of code can be covered by different orderings of the concurrent events in a system, which may result in a different program behavior (illustrated in Section 2).

Mazurkiewicz traces. A foundational formalization of a concurrent system's possible executions is Mazurkiewicz traces [49]. Traces partition the executions of a concurrent system into a set of equivalence classes (traces) based on the orderings of the concurrent events in an execution. Traces are defined on a set of events and an independence relation, where the independent events are commutative; hence, their order in execution does not affect the program behavior. The executions belonging to the same trace order the dependent events in the same total order, but they can reorder

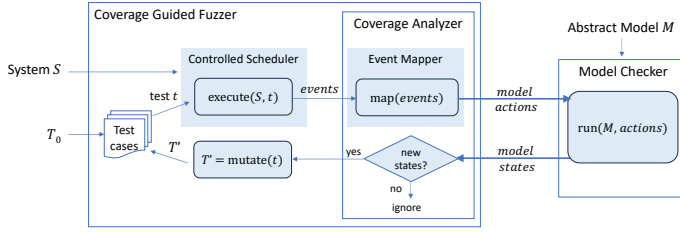


Fig. 2. The workflow of model-guided fuzzing

the independent events. Hence, traces partition the event sequences based on their partial order on the dependent events.

While traces provide a precise formalization for defining the notion of equivalence of executions, defining test coverage classes based on traces is **too fine-grained**, as many different traces may exhibit the same behavior from the perspective of the protocol (illustrated in Section 2).

Coverage of abstract model states. Our method uses the coverage of states in the system’s abstract formal model (e.g., written in TLA+ [39]) to assess the coverage of new system behaviors and guide the test executions. The states in the abstract model succinctly represent the system’s state, abstracting away uninteresting implementation-level details while keeping the essential information of relevant system states and actions. Furthermore, high coverage over the different model states requires exploring all code paths that handle the different system states. We show in our evaluation that high coverage of the formal system states does not degrade the structural code coverage.

We consider the formal model of a system as a labeled transition system $M = \langle Q, I, A, \delta \rangle$ where Q is a set of states, I is the set of initial states, A is the set of actions, and $\delta \subseteq Q \times A \times Q$ is a set of transitions. An action $a \in A$ is enabled at state $q \in Q$ iff $(q, a, q') \in \delta$ for some $q' \in Q$. A run of M is a sequence $\rho = q_0 \xrightarrow{a_1} q_1 \dots \xrightarrow{a_n} q_n$ where $q_0 \in I$ and $(q_i, a_i, q_{i+1}) \in \delta$ holds for all i .

Given an execution of a system S with a set of events $Events$, the formal model of that system M , and the mapping $\phi : Events \mapsto A$, the execution t covers the abstract system states $q_0, \dots, q_n$ iff the sequence of actions $\phi(e_1), \dots, \phi(e_n)$ produce the run $\rho = q_0 \xrightarrow{\phi(e_1)} q_1 \dots \xrightarrow{\phi(e_n)} q_n$ of M .

4 Implementation of ModelFuzz

4.1 The Workflow

Figure 2 illustrates the workflow of model-guided fuzzing. Given a distributed system under test S and a set of initial tests T_0 , the fuzzer runs a test t that represents an event schedule, collects the sequence of executed system events, and maps them to a sequence of actions in the system’s formal model. The resulting model actions are run on the system’s abstract model M . The covered set of abstract model states is used as feedback to guide the test generation.

Different from traditional fuzzers, model-guided fuzzing uses a *controlled scheduler* to control the execution order of events in a distributed system, an *event mapper* to map the system events to abstract actions in the formal model, and a *controlled model checker* to run the given sequence of abstract actions on the model.

Controlled scheduler. The controlled scheduler controls the nondeterminism in the delivery order of messages and executes a particular schedule of events. For each event in an event schedule, $t = \langle \text{buff}_0 : a_0 \rangle \dots \langle \text{buff}_n : a_n \rangle$ with $a_0, \dots, a_n \in \{\text{deliver, crash, restart}\}$, the controlled scheduler either delivers a message to its recipient process, or crashes a process, or restarts a process. To enforce the delivery of a message at a certain time in execution, the scheduler intercepts all in-flight

messages in the system and collects them in the message buffers for each process. For the process crash and restart actions, we provide scripts for starting and stopping the processes in the system under test. The controlled scheduler runs these scripts to enforce a process crash or restart.

Event mapper. The event mapper translates the sequence of concrete events executed on the system by the controlled scheduler into a sequence of actions in the abstract model. Since the implementation level events recorded by the controlled scheduler are specific to the implementation under test, they cannot directly be mapped onto the abstract actions on the formal model. ModelFuzz uses an event mapping interface that the developer implements to map the implementation-level events to the abstract model actions. The mapping of the events from the implementation to the model is mostly syntactic, and it does not require an in-depth understanding of the system under test implementation. For example, event mapping for Raft matches the protocol verbs and fields in the collected messages and the TLA+ model actions (e.g., matching the `AppendEntries` messages with the receiver process, term, and index numbers).

Our implementation (1) converts the collected implementation-level events to a standard JSON encoding and (2) maps the standardized events in JSON format to abstract actions on the model. The second step is embedded in the controlled TLC model checker. The interface to the controlled model checker accepts a sequence of standardized events and outputs the sequence of abstract states observed in the model.

Controlled model checker. The sequence of mapped actions is run on the controlled model checker, which enforces their execution on the model. Unlike a standard model checker, which explores the whole state space of the abstract model, the controlled model checker runs a given sequence of actions and returns the visited set of model states as coverage information. The actions in the specifications used in our evaluation are deterministic, and each execution of the implementation matches a single path in the abstract model. For the models where a single action can lead to multiple model states, model coverage provides an overapproximation, e.g., tracing all paths and collecting all states that could be visited with the executed event schedule.

For the controlled model checker, we implement a simulation engine that (i) controls the next action to take at each state of the model execution and (ii) records the visited states to provide them as feedback to the fuzzer. Our implementation uses TLA+ [39] models of the distributed systems for coverage feedback, and we implement the controlled model checker for the TLC explicit state model checker [72] in the TLA+ Toolbox [37].

4.2 Testing Distributed Systems with ModelFuzz

ModelFuzz can be used to test any distributed system that has a TLA+ model. To test an implementation of a distributed system using ModelFuzz, two key steps are required: (i) intercepting the messages of the system under test to control their delivery and enforce the delivery ordering specified by the fuzzing algorithm, and (ii) writing an event mapper to map the system's events into the actions defined in the TLA+ model. Testing different system implementations and using different TLA+ models do not require any modifications for the controlled TLC model checker.

The first requirement for controlling delivery is a common requirement across all controlled concurrency testing algorithms. This is essential to enable the execution of specific event schedules generated by the algorithms. Additionally, ModelFuzz requires a TLA+ model of the system as well as an event mapping. TLA+ models are available for popular distributed system protocols, such as Two-Phase Commit, Paxos, and Raft. The event mapping, which maps the implementation events to the abstract TLA+ model actions, is mostly synthetic. It is tailored to the specific system being tested and is provided by the developers.

In the rest of this section, we explain our implementation for testing Raft protocol implementations, providing an overview of the protocol, its abstract model, and the event mapper.

Raft protocol. Raft [54] is a leader-based consensus protocol that proceeds in a sequence of terms. Each term begins with a leader election phase. A *candidate* sends RequestVote messages requesting votes from other processes to be elected as *leader*. A candidate process transitions to the *leader* state if it receives a quorum of accepted votes. Upon successful election, the leader executes client requests, records the executed operations in a log, and replicates the log of operations on the other processes by sending AppendEntries messages to the follower processes. The leader also periodically sends Heartbeat messages to communicate its availability to other processes. If a process does not receive Heartbeat messages from the leader over a time period, it moves to a new term, incrementing its term number, and transitions to the *candidate* state.

Abstract protocol model. We use the TLA+ model of the Raft protocol made available by the protocol's authors [28] and extend it by modeling (i) crash and restart of the processes and (ii) snapshot operations. Therefore, besides the variables for the internal states of each process (e.g., the term number, its log of requests), the extended model uses an additional variable to keep the set of active processes in a cluster and introduces crash and restart actions. The crash and restart actions are enabled for the active and crashed processes, respectively, and they update the set of active processes in the cluster. Based on empirical evidence that implementation bugs can occur in the snapshot processing logic, we also extend the model to capture snapshot operations so that the model can guide the executions toward executions where processes trigger snapshots and restore them upon recovery. While the existing TLA+ model does not capture snapshot operations, our model uses a snapshot index variable for each process to record the snapshot index of each process and introduces actions to update the processes' snapshot indices. Our extended TLA+ model³ satisfies the correctness specifications listed in the original model and, therefore, preserves correctness.

Collection of abstract states. While the abstraction provided by the TLA+ model is useful in guiding the fuzzer, we observe the need for further abstracting the set of observed states to guide the exploration to 'interesting' parts of the state space. Concretely, the states of the existing TLA+ model are defined by the local states (e.g., operation logs) of each process, along with the current term numbers of each process. Therefore, the model states differentiate between the system states with the same set of local process states if they are reached in different term numbers. Consider the leader election phase of an execution. Although the local states of the processes do not change, unsuccessful leader election rounds result in hitting new system states since the processes increment their term numbers. Such state information guides the fuzzer toward exploring executions with growing term numbers without covering interesting system behavior.

To guide the fuzzer with more concise state information, we abstract the term numbers in the formal model state. Specifically, we merge two consecutive states in an execution, which differ only in terms of the number of non-leaders.

Note that state abstraction does not require any modification of the formal TLA+ model or the model checker. The abstraction post-processes the collected abstract states from the model checker, and the abstracted states are communicated to the fuzzer. In general, any abstraction can be incorporated by writing an abstraction function from the output state of the model checker to the input state of ModelFuzz.

³https://github.com/burcuku/tlc-controlled-with-benchmarks/blob/main/tla-benchmarks/Raft/model/raft_enhanced.tla

Event mapper from Raft implementations to the abstract TLA+ actions. We implemented event mappers for two different implementations of the Raft protocol (in etcd and Redis) to the abstract TLA+ model. Our mapper for Raft maps system events collected during the execution into three classes of actions in the TLA+ model:

- The cluster actions AddProcess, Crash, Restart, identified with a **process id** as an argument: The system events for adding a process, crashing a process, and restarting a process map to the given actions, respectively.
- The protocol actions of a process, Timeout, ElectLeader, UpdateSnapshotIndex, identified with a **process id and protocol arguments (e.g., term and snapshot numbers)**: The system events to initiate a term change, electing a leader, and updating a snapshot map to the actions, respectively.
- The protocol actions for processing protocol messages, ClientRequest, HandleRequestVoteRequest, HandleRequestVoteResponse, HandleAppendEntriesRequest, HandleAppendEntriesResponse, and HandleNilAppendEntriesResponse, identified with **protocol arguments (e.g., term and index numbers)**: The system events of exchanging these messages with corresponding arguments map to the actions.

5 Experimental Evaluation

We implement ModelFuzz to test two industrial implementations of the Raft protocol [54]: Etcd-raft⁴ and RedisRaft⁵ along with an implementation of the Two-Phase Commit Protocol and an implementation of a parametrized version of the example system presented in Section 2 in the Coyote framework [17]. Our implementation for the Raft protocol uses the TLA+ model of the protocol made available by the protocol's authors [28] and extends it⁶ by modeling (i) crash and restart of the processes and (ii) snapshot operations. Our implementation for the Two-Phase Commit protocol is based on the protocol's model available in the TLA+ documentation [38] and extends it⁷ with transaction variables and support for multiple requests.

We evaluate the performance of model guidance compared to pure (unguided) random testing, guided testing using structural code coverage and trace coverage information, and, finally, state-of-the-art reinforcement learning guided testing, BonusMaxRL [10]. For comparison against BonusMaxRL, we leverage the publicly available implementation^{8,9} and retain the configurations detailed in the original paper.

We evaluate the performance of ModelFuzz in terms of *test coverage* and *bug finding ability*, answering the following research questions:

RQ1 How does the test coverage of ModelFuzz compare to other strategies?

RQ2 Is ModelFuzz more effective at detecting bugs than the other strategies?

We address **RQ1** by comparing the abstract state coverage of ModelFuzz to pure random, line coverage-guided, and trace coverage-guided fuzzing strategies. We address **RQ2** by evaluating the bug-finding effectiveness of different testing strategies in two measures [9]: the unique number of bugs found and the number of test executions to find a bug.

Test oracle. We check the correctness of test executions by checking for assertion violations, exceptions, and crashes. We also check the serializability of the operations in etcd and Redis,

⁴<https://github.com/etcd-io/raft>

⁵<https://github.com/RedisLabs/redisraft>

⁶https://github.com/burcuk/tlc-controlled-with-benchmarks/blob/main/tla-benchmarks/Raft/model/raft_enhanced.tla

⁷<https://github.com/egeberkaygulcan/2PC-Fuzzing/blob/main/tla/TPCL.tla>

⁸<https://github.com/zeu5/raft-rl-test>

⁹<https://github.com/zeu5/dist-rl-testing>

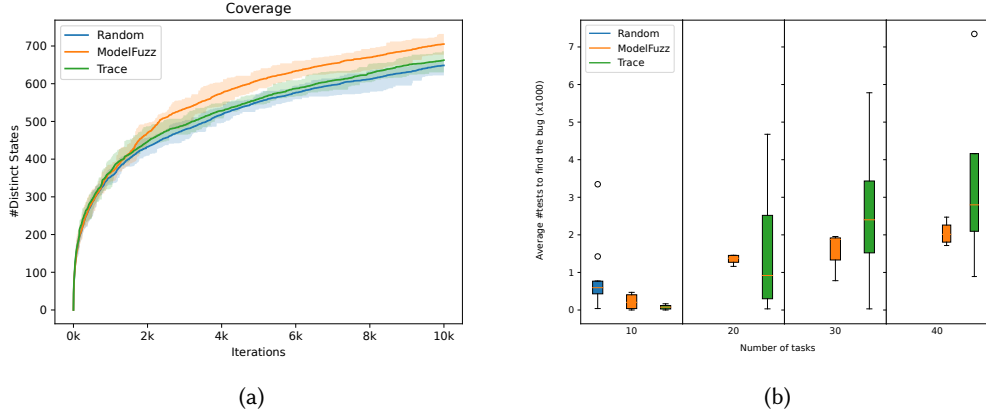


Fig. 3. Testing the microbenchmark. (a) Test coverage for $m = 6$ workers and $n = 40$ tasks (b) #tests to find the bug for varying #tasks with $m = 6$ workers.

running Elle [2], a black-box isolation checker on the collected history of log operations. on the executed operation history.

Statistical evaluation [4]. We analyze the statistical significance of the coverage results of our experiments by performing the Mann-Whitney U-test [48]. We assess ModelFuzz’s bug-finding ability compared to the other strategies using Vargha and Delaney’s $\hat{A}_{12}$ statistic [65], with $\hat{A}_{12} = 0.6$ as in previous work [50].

Test configuration. We run the fuzzers with an initial set of $|T_0| = 20$ random test cases. For each test case that covers a new state, we create five new test cases by mutating the original test case. We multiply the number of generated test cases proportionally with the number of new states observed in the test execution. When the set T of tests to explore becomes empty, we repopulate a random set of tests and repeat the cycle until the test budget is exceeded.

We run the experiments on an Intel(R) Xeon(R) CPU E5-2667 v2 machine with 32 cores and with 252GB of RAM.

5.1 Microbenchmark in Coyote

We implemented a parametrized version of the example in Section 2 in the Coyote framework [17]. The implementation parametrizes the system in (i) the number of worker processes and (ii) the number of Execute task messages that need to be processed to handle a request. For (i), we generate m workers that need to register to the AppMaster before AppMaster can process a client request. For (ii), we modify the processing of Execute so that the Worker divides the work into a chain of n number of tasks.

The system’s possible executions involve different interleavings of the Terminate message with the chain of Execute messages sent to the Worker. We seeded a concurrency bug that occurs if Terminate is processed by the Worker just before the last Execute message while processing a client request. The bug gets harder to trigger with increasing m and n since it requires all m workers to register to AppMaster before Request and also to deliver the chain of Execute messages except for the last one before Terminate.¹⁰

¹⁰ Available at <https://github.com/burcuku/coyote-concurrency-testing>

Table 1. Pairwise $\hat{A}_{12}$ statistic results against ModelFuzz for the microbenchmark with varying m and n .

n	$m = 5$				$m = 6$				$m = 7$			
	10	20	30	40	10	20	30	40	10	20	30	40
Random	0.81	1.00	1.00	1.00	0.86	1.00	1.00	1.00	0.71	1.00	1.00	1.00
Trace	0.33	1.55	0.80	0.83	0.32	0.43	0.73	0.75	0.14	0.14	0.37	0.60

We ran the microbenchmark with 10K iterations with varying $m = \{5, 6, 7\}$ workers and $n = \{10, 20, 30, 40\}$ tasks over ten runs for each configuration. For this system, we do not compare with line-based coverage as the existing coverage tools do not integrate with the framework we use.

Coverage. Figure 3a shows the coverage of the abstract states of the microbenchmark with $m = 6$ workers and $n = 40$ tasks, which is representative of different parameter configurations. Since the microbenchmark is a small example with a small state space, the difference in the explored number of unique abstract states among different testing approaches is not large. However, the results show ModelFuzz’s ability to cover more abstract states compared to random testing and trace coverage guidance. Our Mann-Whitney U-tests show that ModelFuzz achieves statistically significantly better coverage results at $\alpha = 0.05$, compared to random testing and trace-guided fuzzing with p-values $\{0.0001, 0.0004\}$.

Bug finding. We observe a trend with ModelFuzz where it consistently detects the bug faster than random and trace coverage guided testing approaches with increasing m and n . Figure 3b plots the number of test iterations to find the bug for increasing n number of task messages with a fixed $m = 6$ processes. The results show that the increasing number of task messages makes the bug more difficult to detect as it is triggered deep in the execution space. This effect is most evidently seen with pure random testing, as it fails to detect the bug after $n = 10$. Trace coverage guided testing achieves a more consistent variance among different campaigns. However, we observe a trend in its median value for the first iteration to detect the bug, where it declines as the bug gets harder to detect with increasing n . The performance degradation is not as significant for ModelFuzz, as its median value does not change drastically among the experiments.

We use Vargha and Delaney’s $\hat{A}_{12}$ statistic to analyze the significance of our bug-finding results on all parameter configurations. Table 1 lists the pair-wise $\hat{A}_{12}$ statistic values against ModelFuzz for testing the microbenchmark with varying m workers and n tasks. The results show the statistical significance of 17 out of 24 configurations (highlighted in bold), which indicates that ModelFuzz is more effective than the other testing approaches at finding the bug.

5.2 Two-Phase Commit

The Two-Phase Commit protocol is a popular distributed algorithm that ensures all nodes in a distributed database either commit or abort a transaction in a coordinated manner. It plays a crucial role in maintaining consistency within distributed databases and transaction management systems. We provide a thread-based implementation, where we modify the standard protocol to handle requests as sets of variables written by a transaction¹¹. Each node maintains a lock table, which starts in an unlocked state. When a transaction commit request is received, the node checks the lock status of the variables. If none are locked, the vote is approved; otherwise, the request is aborted.

The implementation omits details such as network communication and storage code, as the benchmark’s primary goal is to observe the protocol’s behavioral changes under different request and server interleavings. This results in a compact server implementation of 350 lines of

¹¹<https://github.com/egeberkaygulcan/2PC-Fuzzing>

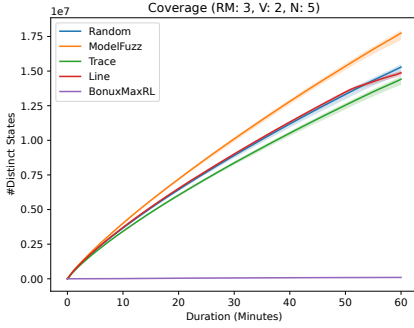


Fig. 4. Test coverage of abstract states of Two-Phase Commit.

Table 2. Average number of abstract states discovered by the strategies.

Guidance	Avg. #states
ModelFuzz	17.74M
Random	15.29M
Trace	14.41M
Line	14.87M
RL	96.98K

Go code, with approximately 150 additional lines for instrumentation. The implementation of ModelFuzz, including the fuzzer loop and communication with the controlled TLC model checker, is approximately 1k lines of Go code.

We tested the Two-Phase Commit protocol using three resource managers (RM), a dedicated transaction manager, two variables (V), and five transaction commit requests (N). These parameters were selected for two key reasons: (i) smaller V values reduce the likelihood of successfully committing concurrent requests, and (ii) the number of distinct abstract model states grows exponentially with increasing N and RM. In experiments with higher N and RM values, we observed similar results, with the only difference being a larger state space requiring a greater time budget for exploration. The chosen parameter set strikes a balance between exploration difficulty and maintaining a manageable state space.

Since the protocol is not fault-tolerant, we excluded crash faults from this benchmark. Each test execution ran for 100 steps per iteration, with a maximum of five messages delivered at each step. We run all tests for one hour and report the average result of 20 test runs.

Coverage. Figure 4 presents the test coverage of the test harnesses, measured by the number of abstract states observed across different strategies. The results show that ModelFuzz surpasses both random and trace coverage-guided exploration strategies. In Figure 2, we report the average number of distinct abstract model states discovered by each strategy, with the best result highlighted in bold. Overall, ModelFuzz observes 1.16x more states than random exploration, 1.23x more than trace coverage-guided exploration, 1.20x more than line coverage-guided exploration, and 182.96x more than the reinforcement learning-based testing BonusMaxRL.

Our analysis of the experiments reveals that BonusMaxRL’s poor performance stems from its high learning cost, which exceeds the execution cost of the program. Within the same time budget, fuzzing-based strategies complete three orders of magnitude more iterations than BonusMaxRL. As a result, BonusMaxRL is significantly less efficient at exploring the state space.

Consistent with our other benchmarks, we address **RQ1** by confirming that model guidance outperforms random exploration, reinforcement learning-guided and other coverage-guidance strategies in the coverage of explored states.

We conduct Mann-Whitney U tests to compare our benchmarks against ModelFuzz and validate the statistical significance of our results. Across 20 experiments, the tests confirm that ModelFuzz discovers significantly more states than the other testing strategies, with p-values of $6.80e-8$ for all comparisons.

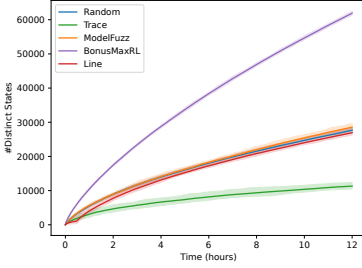


Fig. 5. Test coverage of abstract states.

Table 3. Average number of test iterations to find the synthetic bug. The new bug is only detected by ModelFuzz.

Guidance	Avg. #iterations
ModelFuzz	0.243
Random	0.143
Line	0.322
Trace	0.256
RL	0.234

Table 4. Pairwise $\hat{A}_{12}$ statistics against ModelFuzz for etcd.

Bug	Random	Trace	Line	RL
Seeded	0.57	0.53	0.47	0.49

5.3 Etcd-Raft

Etcd-raft¹² powers the popular distributed key-value store ETCD¹³. It is a well-tested, production-ready implementation of Raft used by companies such as Cloudflare. We instrument its source code¹⁴ to intercept the messages exchanged between processes and implement the fuzzer. The highly modular nature of the ETCD implementation allows us to instrument and run all the nodes within a single OS process, making it easier and faster to start, stop, and restart all the nodes. The Etcd-raft implementation consists of approximately 7k lines of Go code. We instrument its source code and use an additional 1k lines to implement the fuzzer loop, gain control of the messages exchanged between processes, and implement necessary adapters to communicate with the model checker.

We tested the executions of Etcd-raft with three processes and with five client requests. We ran our tests with a crash quota of 10 and delivered a maximum of 5 messages at each step. For each test run, we execute the test for 12 hours, where each iteration runs for a fixed duration of 1s, and report the average results of 20 test runs.

Coverage. Figure 5 reports the test coverage of the test harnesses in the number of abstract states observed with different strategies. The results show that model-guided test generation of ModelFuzz covers similar coverage as random testing (Random), coverage-guided fuzzing using line coverage (Line), and outperforms coverage-guided fuzzing using trace coverage (Trace) in the explored number of unique system states. However, the reinforcement learning approach BonusMaxRL outperforms all other approaches.

Given the nature of our instrumentation with Etcd-raft implementation, ModelFuzz suffers from the performance overhead of communicating with the TLC model checker to measure and retrieve the state trace for the explored execution. However, other approaches are able to achieve better coverage in the absence of this overhead. Comparing model-guided fuzzing and structural guidance, we find that in both cases, the code coverage saturates at 47.9%. Similarly, comparing model-guided fuzzing and trace-based fuzzing, we find that in both cases, we explore 10k unique traces.

¹²<https://github.com/etcd-io/raft>

¹³<https://etcd.io>

¹⁴<https://github.com/zeu5/raft-fuzzing>

Bug finding. Etcd-raft has been the subject of many extensive testing approaches. However, we found one new bug in addition to reproducing a seeded bug. The seeded bug modifies the condition for checking if a process has a quorum of votes. Specifically, we change the valid quorum size from $n/2 + 1$ to $n/3 + 1$. The new bug we found is more subtle and leads to a process crash when accessing a missing snapshot. We reported the bug¹⁵ to developers.

To answer **RQ2**, we analyze the number of detected bugs and the number of test iterations to discover a bug using different strategies. While the seeded bug can be found in each of the 20 trials by all of the strategies, the new bug can only be found using ModelFuzz. Figure 3 reports the average number of iterations to discover the seeded bug using different strategies. The bug can be found by random search faster on average. This can be explained by the characteristics of the bug, which is easily triggered in executions with quorums of only $n/3 + 1$ processes.

Table 4 compares the distributions of the first occurrence of the seeded bug in each trial against ModelFuzz for the different guidance approaches. The Vargha-Delaney ($\hat{A}_{12}$) statistical significance test shows that no approach is significantly better for the seeded bug.

The results show that ModelFuzz is more effective at finding bugs, as only ModelFuzz finds the new bug, and all approaches show comparable performance for the seeded bug.

5.4 RedisRaft

RedisRaft¹⁶ powers the popular high-performance Redis distributed key-value store. It compiles into a module that can be loaded onto the main Redis server, which enables different Redis servers to behave as a group and commit client requests in the same order. We instrumented RedisRaft to intercept the exchanged messages¹⁷ and implement the fuzzer¹⁸. Overall, the Raft module consists of 30k lines of C code. Our instrumentation adds an additional 1.5k lines of C and Go code, where the Go code implements the fuzzer loop. Unlike with Etcd, the instrumentation of RedisRaft requires running all the nodes in different OS processes, along with a network server to capture and deliver messages. Therefore, starting, stopping, and restarting nodes is a time-consuming process.

We tested RedisRaft by running the fuzzer with three processes and five client requests. We ran the test executions with a crash quota of 10 and delivered a maximum of five messages at each step. For each test run, we executed the test for 12 hours, each iteration running for a fixed duration of 3s, and reported the average results of 21 test runs.

Coverage. Figure 6 illustrates the average coverage measures for 12-hour test runs for each testing strategy. We show that ModelFuzz can obtain better coverage over model states compared to random testing, line coverage-guided, and trace coverage-guided fuzzing strategies. The coverage of the reinforcement learning-based approach BonusMaxRL is comparable to ModelFuzz.

On average, ModelFuzz observes 2.58x more states than random exploration, 2.43x more than line coverage, 2.84x more coverage than trace-guided fuzzing, and 1.06x more than BonusMaxRL. We answer **RQ1** with an observation that model guidance outperforms random exploration and other coverage-guidance strategies in the coverage of explored system states and achieves comparable coverage to the BonusMaxRL strategy.

We perform the Mann-Whitney U test over the final coverage numbers to mitigate the effect of randomness. We obtain p values of $3.12e-8$ vs random, trace, and line coverage guidance strategies. The U tests show that ModelFuzz covers significantly more states than the other traditional guidance strategies, barring the reinforcement learning guided approach.

¹⁵<https://github.com/etcd-io/raft/issues/108>

¹⁶<https://github.com/RedisLabs/redisraft>

¹⁷<https://github.com/egeberkaygulcan/redisraft-fuzzing>

¹⁸<https://github.com/zeu5/redisraft-fuzzing>

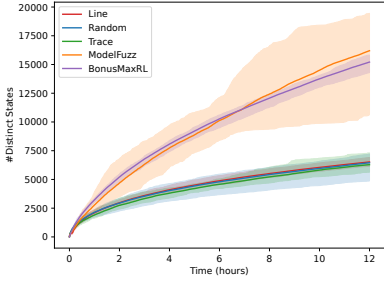


Fig. 6. Test coverage of abstract states.

We also analyze the branch coverage of the tests and observe that guiding the test executions using model coverage does not degrade the coverage over traditional code coverage metrics. Table ?? reports the mean and standard deviation branch coverage of the different guidance methods. We measure the branch coverage of the source code in C using the gcov tool. However, we observe a high variance in the data, which we attribute to the coverage measurement tool. As we run multiple copies of the source code during each test iteration, the branch coverage is combined for each copy. However, we found that the gcov tool does not always merge the coverage values of concurrent invocations correctly. Failing to merge coverage information results in the high variance we observe with the branch coverage measures presented in the table.

Unlike for Etdc-raft, ModelFuzz is able to cover more states when compared to other approaches. The difference lies in the nature of the instrumentation. The performance overhead (in time) of communicating with TLC to obtain coverage information is insignificant compared to the time required to restart all the nodes. Therefore, all approaches cover a similar number of iterations, and we are able to achieve better coverage.

Bug finding. Our experiments for testing RedisRaft discovered 13 different bugs, two of which are known bugs reported in RedisRaft’s issue tracker, and the remaining 11 are new, previously unknown bugs. The bugs occur in the existence of process crashes and restarts with certain orderings of events, and they manifest as thrown exceptions or assertion violations. We investigated the bugs and reported them in the issue tracker of the RedisRaft open-source repository.¹⁹ Table 6 briefly describes the new bugs we discovered.

Among all 13 bugs, ModelFuzz found more bugs than other guidance approaches. Specifically, ModelFuzz found 12 bugs, while random exploration found 8, trace-guided found 9, line-guided found 8, and BonusMaxRL found only 6 of them. For each bug and guidance method, the table in Figure 7 lists the number of runs that find the bug (in parenthesis) and the average first occurrence of the bug (in hours) in the successful runs. For each bug, we highlight the lowest average first occurrence in bold. Among the bugs, one of them is found only using ModelFuzz. Overall, ModelFuzz is more consistent in reproducing the bugs, finding them more frequently than other approaches.

For each of the bugs, we calculate Vargha and Delaney’s $\hat{A}_{12}$ statistics to analyze the pairwise statistical significance of the results. Table 8 reports the $\hat{A}_{12}$ statistics against ModelFuzz for each bug, highlighting the results with statistical significance in bold. The analysis shows that ModelFuzz detects the bug {6} statistically significantly faster than all other guidance approaches, {8, 9, 10, 11} statistically significantly faster than some of the approaches, and comparably faster

Table 5. Average branch coverage of 20 runs.

Method	Branch coverage
ModelFuzz	149.14 ± 111.80
Random	141.07 ± 87.36
Trace	151.07 ± 107.94
Line	150.64 ± 97.02

¹⁹<https://github.com/RedisLabs/redisraft/issues> (Issue numbers: #643-#649)

Table 6. The new bugs found in RedisRaft

ID	Bug description
3	Process crashes when restoring the log from a snapshot stored on disk.
4	Process crashes when polling peer connections. Specifically, a segmentation failure is raised when reading the connection information of a peer.
5	After receiving information of a newly added node, the process crashes when setting a flag indicating the node has been successfully added.
6	Redis server crashes when checking for active client connections.
7	Process crashes when updating the log after receiving an AppendEntries message. Specifically occurs when it has to delete existing log entries.
8	Process crashes when sending AppendEntries and reading from a corrupt log.
9	Process crashes when updating the state to follower upon receiving a message from the leader.
10	Process crashes when updating the state to follower upon receiving a message of a higher term.
11	Process fails to update its current term upon receiving AppendEntries with a higher term.
12	Process fails to update its current term upon receiving RequestVote with a higher term.
13	Process fails to update its current term upon receiving RequestVoteResponse with a higher term.

Table 7. The number of RedisRaft tests for the first occurrences of the bugs (in hours) using different guidance strategies.

ID	Random	Trace	Line	RL	ModelFuzz
1	0.12 (21)	0.18(21)	0.15(21)	-	0.14(21)
2	3.72(19)	4.26(20)	4.06(15)	0.02 (1)	4.13(21)
3	0.01(21)	0.01(21)	0.01(21)	0.02(21)	0.02(21)
4	4.84(16)	5.00(13)	3.45 (10)	3.15(3)	3.63(12)
5	1.78(21)	1.35(21)	1.41 (21)	5.91(8)	1.75(21)
6	-	-	-	-	9.8 (1)
7	-	8.76(2)	5.21 (1)	-	-
8	1.94(21)	1.82(21)	0.93 (21)	2.68(19)	1.14(21)
9	5.76(6)	5.05(4)	2.78 (5)	-	4.71(5)
10	-	-	-	0.17 (1)	3.89(1)
11	4.67 (1)	9.60(1)	-	-	11.41(1)
12	-	-	-	-	3.93(1)
13	-	-	-	-	0.51(1)

Table 8. Pairwise $\hat{A}_{12}$ statistic results against ModelFuzz.

ID	Random	Trace	Line	RL
1	0.399	0.467	0.494	1.000
2	0.491	0.540	0.521	0.000
3	0.434	0.506	0.471	0.585
4	0.529	0.604	0.524	0.462
5	0.480	0.435	0.465	0.892
6	1.000	1.000	1.000	1.000
7	-	-	-	-
8	0.628	0.637	0.439	0.667
9	0.583	0.5	1.000	1.000
10	1.000	1.000	1.000	0.000
11	0.000	0.000	1.000	1.000
12	1.000	1.000	1.000	1.000
13	1.000	1.000	1.000	1.000

for the remainder. While this indicates its ability to detect bugs faster, the $\hat{A}_{12}$ results do not draw clear conclusions on the statistical significance.

What is significant is the number of bugs and the occurrences of the bugs found by ModelFuzz compared to other approaches. Specifically, while RL based approach BonusMaxRL achieves comparable coverage over the states, the number of bugs found by BonusMaxRL is significantly lesser than ModelFuzz or other traditional guidance measures. The targeted coverage guided fuzzing approach is able to lead the exploration to corner cases leading to better bug finding ability.

5.5 Comparison to Related Work

5.5.1 Comparison to Mallory. A closely related work to our approach is MALLORY [50], a greybox fuzzer for distributed systems that employs reinforcement learning to guide test generation.

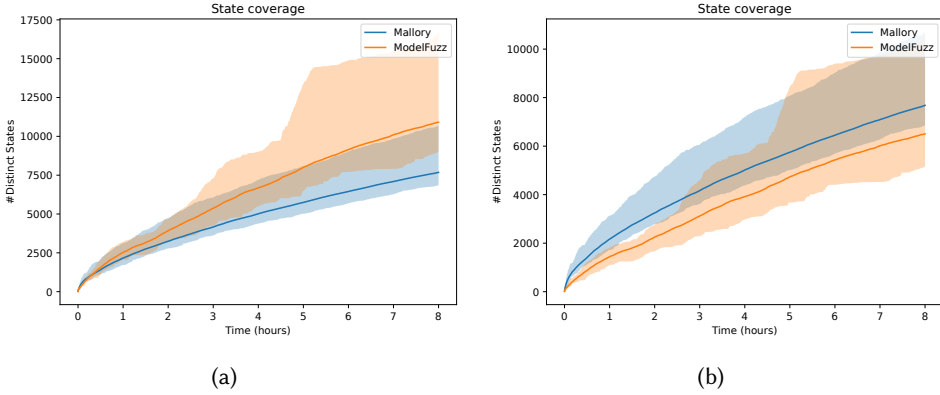


Fig. 7. Comparison of abstract state coverage of Mallory and ModelFuzz for testing Redis for 8-hour test runs with the state abstraction of term numbers (a) including the set of active processes (b) excluding the set of active processes in the abstract system state.

While both MALLORY and ModelFuzz learn from test executions to guide test generation, they adopt different approaches to learning. Mallory runs a long test execution (with a reported duration of 24 hours in their paper) and learns from the exploration within that test execution. After each action taken during the execution, it updates its reinforcement learning policy and makes decisions about subsequent actions based on that information. On the other hand, ModelFuzz runs short test executions (each with a duration of a few seconds) and learns from the exploration across executions. After each test, it analyzes the model coverage achieved and generates the next test cases by mutating the previous executions that have led to coverage of new states.

The two approaches also differ in the coverage information they use to learn from the executions. MALLORY uses a timeline abstraction of the system states, which is similar to the trace coverage notion. However, the set of traces is typically too large for effective guidance. MALLORY overcomes this problem by allowing the user to manually annotate the system's code to specify the interesting parts and use this information in the learning process. On the other hand, ModelFuzz uses the abstract state coverage based on the formal model states (in our experiments, TLA+) of a system.

Our evaluation does not provide an empirical comparison to MALLORY across all the benchmark systems we tested, since the application of MALLORY requires manual inspection and annotation of the source code under test. Therefore, we focus our comparison on the shared benchmark, RedisRaft. Specifically, we compare the *abstract state coverage* and *bug detection* of the two techniques.

We ran MALLORY using the default parameters in its artifact repository, updating its configuration files for the Docker setup and adjusting for differences in our test platforms. Because MALLORY's standard logs lack the detailed event and state information needed to measure abstract state coverage, we extended its instrumentation to RedisRaft to log this information. A limitation in measuring the abstract state coverage of MALLORY tests is due to the length of their test executions. A single test execution of MALLORY processes thousands of user requests, as opposed to five client requests in ModelFuzz tests, making it infeasible to run TLC simulations on the event traces. We mitigate this by collecting abstract state information during the test execution, at the same points where MALLORY collects its event information. Since MALLORY collects the process logs separately, rather than compiling them into a combined serial log of system events like ModelFuzz does, we merged the process logs of MALLORY executions, ordering them by their timestamps. This allowed us to reconstruct the sequence of system states explored during execution. We read the logs into the same definition of model state in MALLORY as we used for ModelFuzz.

State coverage. We ran MALLORY and ModelFuzz for an 8-hour duration of testing, repeated the tests 20 times, and reported their averages. To ensure a fair comparison in the number of requests processed by MALLORY and ModelFuzz, we normalized the logs so that the processes process the same maximum number of user requests simultaneously in the executions of both methods.

The coverage results showed a linear increase in MALLORY's state coverage over the duration of fuzzing. Upon analyzing the covered states, we observed that the linear growth in coverage is introduced by the increasing term numbers during the protocol execution. The term number is a monotonically increasing counter that tracks the leader election processes in the protocol execution. During the 8-hour test execution of MALLORY, the term number increases with the test execution time, while it remains relatively constant, within a few seconds of ModelFuzz tests. To eliminate the effect of the term counter in counting distinct states, we abstracted the term numbers in the state information by considering two states that differ only in the term number as equivalent.

Figure 7a presents the abstract state coverage of MALLORY and ModelFuzz with the term number abstraction. The results show that ModelFuzz and MALLORY perform comparably, with ModelFuzz achieving higher coverage. Figure 7b presents the state coverage with an additional abstraction of the information of *the set of currently active processes* in the system state. As given in Section 4.2, we extend Raft's TLA+ model to track the information of crash and restart of the processes and keep that information as part of the abstract system state. ModelFuzz utilizes this information to guide the new tests through the controlled injection of process crashes. However, the exact points of process crashes and restarts are not available in the MALLORY logs and, therefore, are not read into the abstract states. To present a fair comparison based on the same set of collected data, we additionally present the state coverage results without this information in Figure 7b. The results show comparable performance for MALLORY and ModelFuzz.

Bug finding. ModelFuzz tests replicate 2 of the previously known bugs found by both JEPSEN and MALLORY, and we find 6 new bugs in RedisRaft that were previously not found (described in Table 6). The existence of bugs that could only be detected by MALLORY or only by ModelFuzz suggests the complementary bug-finding capabilities of the two methods. ModelFuzz may miss bugs in parts of the implementation (e.g., rolling back a snapshot) that are not captured by the system's abstract model used. On the other hand, it can uncover bugs that occur in the parts of the system code that are not user-annotated but captured in the model.

Overall, MALLORY and ModelFuzz show comparable performance in abstract state coverage and bug detection, and they can be used complementarily to each other. The performance of MALLORY is influenced by user annotations that identify significant states, while ModelFuzz relies on the expressiveness of the abstract model used. Depending on the user annotations for MALLORY and the abstract model used by ModelFuzz, they can be more effective at covering and detecting bugs in the complementary parts of the execution space.

5.5.2 Comparison to BonusMaxRL. We empirically compare our approach against BonusMaxRL [10], which utilizes reinforcement learning techniques to explore the state space of a distributed system. Specifically, BonusMaxRL provides a novel reward mechanism that extends prior work and has been proven to be successful in finding bugs in both RedisRaft and Etcd-raft benchmarks. We present empirical comparison of the coverage for all our benchmarks. For the Etcd benchmark, BonusMaxRL outperforms ModelFuzz while for the RedisRaft benchmark, BonusMaxRL achieves comparable coverage to ModelFuzz. However, we observe that BonusMaxRL fails to find all the bugs in both benchmarks.

5.6 Summary of the Evaluation Results

In summary, we answer **RQ1**, showing that ModelFuzz outperforms pure random testing and code-coverage and trace-guided fuzzing strategies in the coverage of model states, yet producing coverage comparable to or worse than BonusMaxRL. We answered **RQ2** by comparing the number of test executions until the first occurrence of a bug. We showed that ModelFuzz discovers bugs faster than the other strategies on average.

Moreover, the intuition of using model state coverage to guide the fuzzer has shown effective, discovering 1 new bug for Etcd-raft and 11 new bugs for RedisRaft, along with replicating 2 previously known bugs. Note that 2 of the new bugs in RedisRaft discovered by ModelFuzz could not be detected by other approaches in the given test budget, showing that our approach is more robust in finding bugs earlier than other guidance approaches.

Finally, our evaluation on several systems under test that implement different distributed protocols demonstrates the applicability of ModelFuzz across multiple systems and the generalizability of the findings.

6 Limitations

Model-guided fuzzing requires the model to be close to the implementation so that the model provides effective feedback for test generation. The abstraction level of the model heavily affects the performance of model guidance. A high-level model abstracting away critical aspects will be ineffective at guiding the tests toward exploring these aspects in the system implementation. For example, while the TLA+ model of Raft protocol [28] provided by the protocol's authors models process interactions, it does not capture process crash and restart actions. Therefore, using this model to guide test generation will be ineffective at directing the test executions with process crashes and restarts that potentially lead to previously unseen executions. Conversely, a model that is too fine-grained and captures all implementation details is also not helpful. For example, a model that implements HeartBeat messages or captures too many details of process states distinguishes equivalent high-level behaviors of the system. It can guide test generation toward exploring different executions that produce equivalent system behavior.

7 Related Work

Coverage-guided fuzzing. Fuzzing [6, 8, 32, 43, 47, 75] has been extensively studied for test input generation for sequential programs. Rather than generating test inputs independently at random, coverage-guided fuzzing techniques incorporate lightweight program instrumentation to collect some feedback information from the test executions and use that information to guide the generation of new tests. Recent techniques target the generation of different types of program inputs [22, 29, 63] and improve fuzzer performance [7, 61]. Extensions of AFL [74] such as AFLNET [56], StateAFL [52] test communication protocols by mutating structured message inputs guided by the states explored. Different from test input fuzzing, our work utilizes the fuzzing approach to generate event schedules for testing distributed system executions.

Testing distributed systems. Distributed systems have been the focus of a wide range of research such as systematic testing and model checking [18, 20, 23, 41, 42, 57, 62, 70], random fault injection [34] or scheduling [27], and designing methods for carefully sampling event schedules and faults [14, 21, 35, 36, 68, 73]. While some methods guide fault injection using lineage [3], runtime state [12], or system's meta-variables [46], most techniques do not exploit information observed in the test executions and generate tests independently from each other.

Fuzzing concurrent and distributed systems. Fuzzing methods for multithreaded concurrency guide the tests by monitoring races and synchronization events [60, 66, 71], execution states caused by thread interleavings [13], coverage of concurrent call pairs [33], and using the reads-from relation between the memory accesses [69]. These methods are designed for multithreaded programs, and they do not target distributed concurrency.

Recent testing techniques for distributed systems learn from the explored executions and adapt reinforcement learning or fuzzing approaches to generate new tests. QL [51] uses reinforcement learning to guide the exploration to unexplored parts of the execution space. Recent work [10] improves the reward augmentation using theoretical results in reinforcement learning and programmer-provided information for the interesting parts of the search space. Evolutionary search-based testing [64] directs the exploration toward a fitness function.

Existing work for fuzzing distributed systems uses code coverage and message traces as program feedback. CrashFuzz [26] and FaultFuzz[24] fault-injection tools inject process crashes using code coverage information as program feedback. MALLORY [50] builds on JEPSEN [34] and guides test generation using programmer annotations that mark interesting parts of the code and an event timeline abstraction (close to traces) as the feedback information. Different from using user annotations, code coverage, or trace coverage for guidance, ModelFuzz uses guidance from the abstract system model.

Model-based testing. Model-based testing uses formal models (e.g., TLA+ specifications) to exhaustively enumerate system executions and enforce them on the implementation. Its applications include testing APIs [5], fragments of HTTP protocol [44] and specific systems [59]. Protocol fuzzers DTLS-Fuzzer [25] and EDHOC-Fuzzer [58] use model learning to generate a state machine model of the protocol implementations which can be used for model-based testing.

MOCKET [67] and MET [76] adopt model-based testing to test distributed systems and CRDTs. MOCKET uses the paths in the model's state space graph as test cases, and it enforces the system under test to run the actions generated on the system's model on the corresponding states and actions in the implementation. For this, it synchronizes the executions of the system and the model at each step. The synchronization of executions requires heavy annotation on the system's source code to mark the program variables and messages associated with the system's model variables.

Our work conceptually differs from model-based testing, as it performs an unconstrained exploration of the implementation. Model-based testing generates test cases using model paths, and hence, it does not cover parts of the implementation that are abstracted away in the model. In contrast, model-guided fuzzing explores the executions of the implementation, including those not captured by the model.

Recent work [15] uses TLA+ models to validate execution traces of distributed systems. It uses programmer instrumentation on the system implementation to identify the points of state variable updates and transitions and validates the collected execution traces on the TLA+ model to detect the conformance or discrepancies between the system specification and its implementations. While this work focuses on verifying execution traces, its future work points to using TLA+ models for guiding test executions towards interesting parts of the search space, as we do with ModelFuzz.

Model-guided fuzzing. ModelFuzz stands out from the state-of-the-art in testing distributed systems as it steers the exploration towards more coverage of system behaviors without the need for the programmer's comprehension and annotations on the source code (e.g., to identify and mark interesting parts of the source code or using heavy instrumentation to synchronize its execution with a formal model). Instead, it utilizes the system's abstract model, which already captures the essential information about system behavior, to collect information about the explored system behaviors and generate new tests.

8 Conclusion

In this paper, we present ModelFuzz, a new approach for fuzzing distributed systems implementations. Our novel approach uses coverage over abstract model states as feedback for the fuzzer to generate test executions and guide the test generation toward interesting parts of the state space. We use ModelFuzz to test the implementations of different distributed protocols, including two production distributed system implementations. We show that ModelFuzz can achieve high model coverage, allowing us to discover new bugs and replicate known bugs more quickly than other guidance methods.

Data Availability Statement

The implementation of ModelFuzz and the systems under test are available in the accompanying artifact [31].

Acknowledgments

We would like to thank the anonymous reviewers for their constructive comments and suggestions. We would also like to thank Ruijie Meng for her help with running the Mallory tool. This work was partially supported by the Amazon Research Award.

References

- [1] Gul A. Agha. 1985. *Actors: a Model of Concurrent Computation in Distributed Systems (Parallel Processing, Semantics, Open, Programming Languages, Artificial Intelligence)*. Ph. D. Dissertation. University of Michigan, USA.
- [2] Peter Alvaro and Kyle Kingsbury. 2020. Elle: Inferring Isolation Anomalies from Experimental Observations. *Proc. VLDB Endow.* 14, 3 (2020), 268–280. doi:10.5555/3430915.3442427
- [3] Peter Alvaro, Joshua Rosen, and Joseph M. Hellerstein. 2015. Lineage-driven Fault Injection. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data, Melbourne, Victoria, Australia, May 31 - June 4, 2015*, Timos K. Sellis, Susan B. Davidson, and Zachary G. Ives (Eds.). ACM, 331–346. doi:10.1145/2723372.2723711
- [4] Andrea Arcuri and Lionel C. Briand. 2014. A Hitchhiker’s guide to statistical tests for assessing randomized algorithms in software engineering. *Softw. Test. Verification Reliab.* 24, 3 (2014), 219–250. doi:10.1002/STVR.1486
- [5] Cyrille Valentin Artho, Armin Biere, Masami Hagiya, Eric Platon, Martina Seidl, Yoshinori Tanabe, and Mitsuharu Yamamoto. 2013. Modbat: A Model-Based API Tester for Event-Driven Systems. In *Hardware and Software: Verification and Testing - 9th International Haifa Verification Conference, HVC 2013, Haifa, Israel, November 5-7, 2013, Proceedings (Lecture Notes in Computer Science, Vol. 8244)*, Valeria Bertacco and Axel Legay (Eds.). Springer, 112–128. doi:10.1007/978-3-319-03077-7_8
- [6] Jinsheng Ba, Marcel Böhme, Zahra Mirzamomen, and Abhik Roychoudhury. 2022. Stateful Greybox Fuzzing. In *31st USENIX Security Symposium, USENIX Security 2022, Boston, MA, USA, August 10-12, 2022*, Kevin R. B. Butler and Kurt Thomas (Eds.). USENIX Association, 3255–3272.
- [7] Marcel Böhme, Valentin J. M. Manès, and Sang Kil Cha. 2020. Boosting fuzzer efficiency: an information theoretic perspective. In *ESEC/FSE ’20: 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Virtual Event, USA, November 8-13, 2020*, Prem Devanbu, Myra B. Cohen, and Thomas Zimmermann (Eds.). ACM, 678–689. doi:10.1145/3368089.3409748
- [8] Marcel Böhme, Van-Thuan Pham, and Abhik Roychoudhury. 2016. Coverage-based Greybox Fuzzing as Markov Chain. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, Vienna, Austria, October 24-28, 2016*, Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi (Eds.). ACM, 1032–1043. doi:10.1145/2976749.2978428
- [9] Marcel Böhme, László Szekeres, and Jonathan Metzman. 2022. On the Reliability of Coverage-Based Fuzzer Benchmarking. In *44th IEEE/ACM 44th International Conference on Software Engineering, ICSE 2022, Pittsburgh, PA, USA, May 25-27, 2022*. ACM, 1621–1633. doi:10.1145/3510003.3510230
- [10] Andrea Borgarelli, Constantin Enea, Rupak Majumdar, and Srinidhi Nagendra. 2024. Reward Augmentation in Reinforcement Learning for Testing Distributed Systems. *Proceedings of the ACM on Programming Languages* 8, OOPSLA2 (2024), 1928–1954.
- [11] James Bornholt, Rajeev Joshi, Vytautas Astrauskas, Brendan Cully, Bernhard Kragl, Seth Markle, Kyle Sauri, Drew Schleit, Grant Slatton, Serdar Tasiran, Jacob Van Geffen, and Andrew Warfield. 2021. Using Lightweight Formal Methods to Validate a Key-Value Storage Node in Amazon S3. In *SOSP ’21: ACM SIGOPS 28th Symposium on Operating*

- Systems Principles, Virtual Event / Koblenz, Germany, October 26-29, 2021*, Robbert van Renesse and Nickolai Zeldovich (Eds.). ACM, 836–850. doi:10.1145/3477132.3483540
- [12] Haicheng Chen, Wensheng Dou, Dong Wang, and Feng Qin. 2020. CoFI: Consistency-Guided Fault Injection for Cloud Systems. In *35th IEEE/ACM International Conference on Automated Software Engineering, ASE 2020, Melbourne, Australia, September 21-25, 2020*. IEEE, 536–547. doi:10.1145/3324884.3416548
 - [13] Hongxu Chen, Shengjian Guo, Yinxing Xue, Yulei Sui, Cen Zhang, Yuekang Li, Haijun Wang, and Yang Liu. 2020. MUZZ: Thread-aware Grey-box Fuzzing for Effective Bug Hunting in Multithreaded Programs. In *29th USENIX Security Symposium, USENIX Security 2020, August 12-14, 2020, Srdjan Capkun and Franziska Roesner (Eds.)*. USENIX Association, 2325–2342.
 - [14] Dmitry Chistikov, Rupak Majumdar, and Filip Nksic. 2016. Hitting Families of Schedules for Asynchronous Programs. In *Computer Aided Verification - 28th International Conference, CAV 2016, Toronto, ON, Canada, July 17-23, 2016, Proceedings, Part II (Lecture Notes in Computer Science, Vol. 9780)*, Swarat Chaudhuri and Azadeh Farzan (Eds.). Springer, 157–176. doi:10.1007/978-3-319-41540-6_9
 - [15] Horatiu Cirstea, Markus A. Kuppe, Benjamin Loillier, and Stephan Merz. 2024. Validating Traces of Distributed Programs Against TLA⁺ Specifications. In *Software Engineering and Formal Methods - 22nd International Conference, SEFM 2024, Aveiro, Portugal, November 6-8, 2024, Proceedings (Lecture Notes in Computer Science, Vol. 15280)*, Alexandre Madeira and Alexander Knapp (Eds.). Springer, 126–143. doi:10.1007/978-3-031-77382-2_8
 - [16] Pantazis Deligiannis, Alastair F. Donaldson, Jeroen Ketema, Akash Lal, and Paul Thomson. 2015. Asynchronous programming, analysis and testing with state machines. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation, Portland, OR, USA, June 15-17, 2015*, David Grove and Stephen M. Blackburn (Eds.). ACM, 154–164. doi:10.1145/2737924.2737996
 - [17] Pantazis Deligiannis, Narayanan Ganapathy, Akash Lal, and Shaz Qadeer. 2021. Building Reliable Cloud Services Using Coyote Actors. In *SoCC '21: ACM Symposium on Cloud Computing, Seattle, WA, USA, November 1 - 4, 2021*, Carlo Curino, Georgia Koutrika, and Ravi Netravali (Eds.). ACM, 108–121. doi:10.1145/3472883.3486983
 - [18] Pantazis Deligiannis, Matt McCutchen, Paul Thomson, Shuo Chen, Alastair F. Donaldson, John Erickson, Cheng Huang, Akash Lal, Rashmi Mudduluru, Shaz Qadeer, and Wolfram Schulte. 2016. Uncovering Bugs in Distributed Storage Systems during Testing (Not in Production!). In *14th USENIX Conference on File and Storage Technologies, FAST 2016, Santa Clara, CA, USA, February 22-25, 2016*, Angela Demke Brown and Florentina I. Popovici (Eds.). USENIX Association, 249–262.
 - [19] Ankush Desai, Vivek Gupta, Ethan K. Jackson, Shaz Qadeer, Sriram K. Rajamani, and Damien Zufferey. 2013. P: safe asynchronous event-driven programming. In *ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI '13, Seattle, WA, USA, June 16-19, 2013*, Hans-Juergen Boehm and Cormac Flanagan (Eds.). ACM, 321–332. doi:10.1145/2491956.2462184
 - [20] Ankush Desai, Shaz Qadeer, and Sanjit A. Seshia. 2015. Systematic testing of asynchronous reactive systems. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering, ESEC/FSE 2015, Bergamo, Italy, August 30 - September 4, 2015*, Elisabetta Di Nitto, Mark Harman, and Patrick Heymans (Eds.). ACM, 73–83. doi:10.1145/2786805.2786861
 - [21] Cezara Dragoi, Constantin Enea, Burcu Kulahcioglu Ozkan, Rupak Majumdar, and Filip Nksic. 2020. Testing consensus implementations using communication closure. *Proc. ACM Program. Lang.* 4, OOPSLA (2020), 210:1–210:29. doi:10.1145/3428278
 - [22] Rafael Dutra, Rahul Gopinath, and Andreas Zeller. 2024. FormatFuzzer: Effective Fuzzing of Binary File Formats. *ACM Trans. Softw. Eng. Methodol.* 33, 2 (2024), 53:1–53:29. doi:10.1145/3628157
 - [23] Constantin Enea, Dimitra Giannakopoulou, Michalis Kokologiannakis, and Rupak Majumdar. 2024. Model Checking Distributed Protocols in Must. *Proceedings of the ACM on Programming Languages* 8, OOPSLA2 (2024), 1900–1927.
 - [24] Wenhan Feng, Qiugen Pei, Yu Gao, Dong Wang, Wensheng Dou, Jun Wei, Zheheng Liang, and Zhenyue Long. 2024. FaultFuzz: A Coverage Guided Fault Injection Tool for Distributed Systems. In *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings, ICSE Companion 2024, Lisbon, Portugal, April 14-20, 2024*. ACM, 129–133. doi:10.1145/3639478.3640036
 - [25] Paul Fiterau-Brosteau, Bengt Jonsson, Konstantinos Sagonas, and Fredrik Tåquist. 2022. DTLS-Fuzzer: A DTLS Protocol State Fuzzer. In *15th IEEE Conference on Software Testing, Verification and Validation, ICST 2022, Valencia, Spain, April 4-14, 2022*. IEEE, 456–458. doi:10.1109/ICST53961.2022.00051
 - [26] Yu Gao, Wensheng Dou, Dong Wang, Wenhan Feng, Jun Wei, Hua Zhong, and Tao Huang. 2023. Coverage Guided Fault Injection for Cloud Systems. In *45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14-20, 2023*. IEEE, 2211–2223. doi:10.1109/ICSE48619.2023.00186
 - [27] GitHub. [n. d.]. Namazu: Programmable Fuzzy Scheduler for Testing Distributed Systems. <https://github.com/osrg/namazu>.
 - [28] Github. [n. d.]. TLA+ specification for the Raft consensus algorithm. <https://github.com/ongardie/raft.tla>.

- [29] Patrice Godefroid, Bo-Yuan Huang, and Marina Polishchuk. 2020. Intelligent REST API data fuzzing. In *ESEC/FSE '20: 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Virtual Event, USA, November 8-13, 2020*, Prem Devanbu, Myra B. Cohen, and Thomas Zimmermann (Eds.). ACM, 725–736. doi:10.1145/3368089.3409719
- [30] Rahul Gopinath, Björn Mathis, and Andreas Zeller. 2020. Mining input grammars from dynamic control flow. In *ESEC/FSE '20: 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Virtual Event, USA, November 8-13, 2020*, Prem Devanbu, Myra B. Cohen, and Thomas Zimmermann (Eds.). ACM, 172–183. doi:10.1145/3368089.3409679
- [31] Ege Berkay Gulcan, Burcu Kulahcioglu Ozkan, Rupak Majumdar, and Srinidhi Nagendra. 2025. *Artifact for Model-guided Fuzzing of Distributed Systems*. doi:10.5281/zenodo.15790701
- [32] Marc Heuse, Heiko Eißfeldt, Andrea Fioraldi, and Dominik Maier. 2022. *AFL++*. <https://github.com/AFLplusplus/AFLplusplus>
- [33] Zu-Ming Jiang, Jia-Ju Bai, Kangjie Lu, and Shi-Min Hu. 2022. Context-Sensitive and Directional Concurrency Fuzzing for Data-Race Detection. In *29th Annual Network and Distributed System Security Symposium, NDSS 2022, San Diego, California, USA, April 24-28, 2022*. The Internet Society.
- [34] Kyle Kingsbury. 2022. Jepsen. <http://jepsen.io/>.
- [35] Burcu Kulahcioglu Ozkan, Rupak Majumdar, Filip Niksic, Mitra Tabaei Befrouei, and Georg Weissenbacher. 2018. Randomized testing of distributed systems with probabilistic guarantees. *Proc. ACM Program. Lang.* 2, OOPSLA (2018), 160:1–160:28. doi:10.1145/3276530
- [36] Burcu Kulahcioglu Ozkan, Rupak Majumdar, and Simin Oraee. 2019. Trace aware random testing for distributed systems. *Proc. ACM Program. Lang.* 3, OOPSLA (2019), 180:1–180:29. doi:10.1145/3360606
- [37] Markus Alexander Kuppe, Leslie Lamport, and Daniel Ricketts. 2019. The TLA+ Toolbox. In *Proceedings Fifth Workshop on Formal Integrated Development Environment, F-IDE@FM 2019, Porto, Portugal, 7th October 2019 (EPTCS, Vol. 310)*, Rosemary Monahan, Virgile Prevosto, and José Proença (Eds.). 50–62. doi:10.4204/EPTCS.310.6
- [38] Leslie Lamport. [n. d.]. The TLA+ Home Page. <https://lamport.azurewebsites.net/tla/tla.html>.
- [39] Leslie Lamport. 2002. *Specifying Systems, The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley.
- [40] Xuan-Bach Dinh Le, Corina S. Pasareanu, Rohan Padhye, David Lo, Willem Visser, and Koushik Sen. 2019. Saffron: Adaptive Grammar-based Fuzzing for Worst-Case Analysis. *ACM SIGSOFT Softw. Eng. Notes* 44, 4 (2019), 14. doi:10.1145/3364452.3364455
- [41] Tanakorn Leesatapornwongsa, Mingzhe Hao, Pallavi Joshi, Jeffrey F. Lukman, and Haryadi S. Gunawi. 2014. SAMC: Semantic-Aware Model Checking for Fast Discovery of Deep Bugs in Cloud Systems. In *11th USENIX Symposium on Operating Systems Design and Implementation, OSDI '14, Broomfield, CO, USA, October 6-8, 2014*, Jason Flinn and Hank Levy (Eds.). USENIX Association, 399–414.
- [42] Tanakorn Leesatapornwongsa, Jeffrey F. Lukman, Shan Lu, and Haryadi S. Gunawi. 2016. TaxDC: A Taxonomy of Non-Deterministic Concurrency Bugs in Datacenter Distributed Systems. In *Proceedings of the Twenty-First International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS 2016, Atlanta, GA, USA, April 2-6, 2016*, Tom Conte and Yuan Yuan Zhou (Eds.). ACM, 517–530. doi:10.1145/2872362.2872374
- [43] Caroline Lemieux and Koushik Sen. 2018. FairFuzz: a targeted mutation strategy for increasing greybox fuzz testing coverage. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering, ASE 2018, Montpellier, France, September 3-7, 2018*, Marianne Huchard, Christian Kästner, and Gordon Fraser (Eds.). ACM, 475–485. doi:10.1145/3238147.3238176
- [44] Yishuai Li, Benjamin C. Pierce, and Steve Zdancewic. 2021. Model-based testing of networked applications. In *ISSTA '21: 30th ACM SIGSOFT International Symposium on Software Testing and Analysis, Virtual Event, Denmark, July 11-17, 2021*, Cristian Cadar and Xiangyu Zhang (Eds.). ACM, 529–539. doi:10.1145/3460319.3464798
- [45] Lightbend, Inc. 2011–2023. *Akka Documentation*. Retrieved August 8, 2023 from <https://akka.io/docs/>
- [46] Jie Lu, Chen Liu, Lian Li, Xiaobing Feng, Feng Tan, Jun Yang, and Liang You. 2019. CrashTuner: detecting crash-recovery bugs in cloud systems via meta-info analysis. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles, SOSP 2019, Huntsville, ON, Canada, October 27-30, 2019*, Tim Brecht and Carey Williamson (Eds.). ACM, 114–130. doi:10.1145/3341301.3359645
- [47] Valentin J. M. Manès, HyungSeok Han, Choongwoo Han, Sang Kil Cha, Manuel Egele, Edward J. Schwartz, and Maverick Woo. 2021. The Art, Science, and Engineering of Fuzzing: A Survey. *IEEE Trans. Software Eng.* 47, 11 (2021), 2312–2331. doi:10.1109/TSE.2019.2946563
- [48] H. B. Mann and D. R. Whitney. 1947. On a Test of Whether one of Two Random Variables is Stochastically Larger than the Other. *The Annals of Mathematical Statistics* 18, 1 (March 1947), 50–60. doi:10.1214/aoms/1177730491
- [49] Antoni W. Mazurkiewicz. 1986. Trace Theory. In *Petri Nets: Central Models and Their Properties, Advances in Petri Nets 1986, Part II, Proceedings of an Advanced Course, Bad Honnef, Germany, 8-19 September 1986 (Lecture Notes in*

- Computer Science*, Vol. 255), Wilfried Brauer, Wolfgang Reisig, and Grzegorz Rozenberg (Eds.). Springer, 279–324. doi:10.1007/3-540-17906-2_30
- [50] Ruijie Meng, George Pirlea, Abhik Roychoudhury, and Ilya Sergey. 2023. Greybox Fuzzing of Distributed Systems. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, CCS 2023, Copenhagen, Denmark, November 26-30, 2023*, Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda (Eds.). ACM, 1615–1629. doi:10.1145/3576915.3623097
 - [51] Suvam Mukherjee, Pantazis Deligiannis, Arpita Biswas, and Akash Lal. 2020. Learning-based controlled concurrency testing. *Proc. ACM Program. Lang.* 4, OOPSLA (2020), 230:1–230:31. doi:10.1145/3428298
 - [52] Roberto Natella. 2022. StateAFL: Greybox fuzzing for stateful network servers. *Empir. Softw. Eng.* 27, 7 (2022), 191. doi:10.1007/s10664-022-10233-3
 - [53] Chris Newcombe, Tim Rath, Fan Zhang, Bogdan Munteanu, Marc Brooker, and Michael Deardeuff. 2015. How Amazon web services uses formal methods. *Commun. ACM* 58, 4 (2015), 66–73. doi:10.1145/2699417
 - [54] Diego Ongaro and John K. Ousterhout. 2014. In Search of an Understandable Consensus Algorithm. In *2014 USENIX Annual Technical Conference, USENIX ATC '14, Philadelphia, PA, USA, June 19-20, 2014*, Garth Gibson and Nickolai Zeldovich (Eds.). USENIX Association, 305–319.
 - [55] Rohan Padhye, Caroline Lemieux, Koushik Sen, Mike Papadakis, and Yves Le Traon. 2019. Semantic fuzzing with zest. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2019, Beijing, China, July 15-19, 2019*, Dongmei Zhang and Anders Möller (Eds.). ACM, 329–340. doi:10.1145/3293882.3330576
 - [56] Van-Thuan Pham, Marcel Böhme, and Abhik Roychoudhury. 2020. AFLNET: A Greybox Fuzzer for Network Protocols. In *13th IEEE International Conference on Software Testing, Validation and Verification, ICST 2020, Porto, Portugal, October 24-28, 2020*. IEEE, 460–465. doi:10.1109/ICST46399.2020.00062
 - [57] Lauren Pick, Ankush Desai, and Aarti Gupta. 2023. Psym: Efficient Symbolic Exploration of Distributed Systems. *Proc. ACM Program. Lang.* 7, PLDI (2023), 660–685. doi:10.1145/3591247
 - [58] Konstantinos Sagonas and Thanasis Typaldos. 2023. EDHOC-Fuzzer: An EDHOC Protocol State Fuzzer. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2023, Seattle, WA, USA, July 17-21, 2023*, René Just and Gordon Fraser (Eds.). ACM, 1495–1498. doi:10.1145/3597926.3604922
 - [59] Judah Schvimer, A. Jesse Jiryu Davis, and Max Hirschhorn. 2020. eXtreme Modelling in Practice. *Proc. VLDB Endow.* 13, 9 (2020), 1346–1358. doi:10.14778/3397230.3397233
 - [60] Koushik Sen. 2008. Race directed random testing of concurrent programs. In *Proceedings of the ACM SIGPLAN 2008 Conference on Programming Language Design and Implementation, Tucson, AZ, USA, June 7-13, 2008*, Rajiv Gupta and Saman P. Amarasinghe (Eds.). ACM, 11–21. doi:10.1145/1375581.1375584
 - [61] Chaofan Shou, Shangyin Tan, and Koushik Sen. 2023. ItyFuzz: Snapshot-Based Fuzzer for Smart Contract. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2023, Seattle, WA, USA, July 17-21, 2023*, René Just and Gordon Fraser (Eds.). ACM, 322–333. doi:10.1145/3597926.3598059
 - [62] Jiri Simsa, Randy Bryant, and Garth A. Gibson. 2011. dBug: Systematic Testing of Unmodified Distributed and Multi-threaded Systems. In *Model Checking Software - 18th International SPIN Workshop, Snowbird, UT, USA, July 14-15, 2011. Proceedings (Lecture Notes in Computer Science, Vol. 6823)*, Alex Groce and Madanlal Musuvathi (Eds.). Springer, 188–193. doi:10.1007/978-3-642-22306-8_14
 - [63] Dominic Steinhöfel and Andreas Zeller. 2022. Input invariants. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2022, Singapore, Singapore, November 14-18, 2022*, Abhik Roychoudhury, Cristian Cadar, and Miryung Kim (Eds.). ACM, 583–594. doi:10.1145/3540250.3549139
 - [64] Martijn van Meerten, Burcu Kulahcioglu Ozkan, and Annibale Panichella. 2023. Evolutionary Approach for Concurrency Testing of Ripple Blockchain Consensus Algorithm. In *45th IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice, SEIP@ICSE 2023, Melbourne, Australia, May 14-20, 2023*. IEEE, 36–47. doi:10.1109/ICSE-SEIP58684.2023.00009
 - [65] András Vargha and Harold D. Delaney. 2000. A Critique and Improvement of the CL Common Language Effect Size Statistics of McGraw and Wong. *Journal of Educational and Behavioral Statistics* 25, 2 (2000), 101–132. arXiv:https://doi.org/10.3102/10769986025002101 doi:10.3102/10769986025002101
 - [66] Chao Wang, Mahmoud Said, and Aarti Gupta. 2011. Coverage guided systematic concurrency testing. In *Proceedings of the 33rd International Conference on Software Engineering, ICSE 2011, Waikiki, Honolulu, HI, USA, May 21-28, 2011*, Richard N. Taylor, Harald C. Gall, and Nenad Medvidovic (Eds.). ACM, 221–230. doi:10.1145/1985793.1985824
 - [67] Dong Wang, Wensheng Dou, Yu Gao, Chenao Wu, Jun Wei, and Tao Huang. 2023. Model Checking Guided Testing for Distributed Systems. In *Proceedings of the Eighteenth European Conference on Computer Systems, EuroSys 2023, Rome, Italy, May 8-12, 2023*, Giuseppe Antonio Di Luna, Leonardo Querzoni, Alexandra Fedorova, and Dushyanth Narayanan (Eds.). ACM, 127–143. doi:10.1145/3552326.3587442

- [68] Levin N. Winter, Florena Buse, Daan de Graaf, Klaus von Gleissenthall, and Burcu Kulahcioglu Ozkan. 2023. Randomized Testing of Byzantine Fault Tolerant Algorithms. *Proc. ACM Program. Lang.* 7, OOPSLA1 (2023), 757–788. doi:10.1145/3586053
- [69] Dylan Wolff, ShiZheng, Gregory Duck, Umang Mathur, and Abhik Roychoudhury. [n. d.]. Greybox Fuzzing for Concurrency Testing. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS 2024, San Diego, USA, April 27 - May 1, 2024*. to appear.
- [70] Junfeng Yang, Tisheng Chen, Ming Wu, Zhilei Xu, Xuezheng Liu, Haoxiang Lin, Mao Yang, Fan Long, Lintao Zhang, and Lidong Zhou. 2009. MODIST: Transparent Model Checking of Unmodified Distributed Systems. In *Proceedings of the 6th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2009, April 22-24, 2009, Boston, MA, USA*, Jennifer Rexford and Emin Gün Sirer (Eds.). USENIX Association, 213–228.
- [71] Jie Yu, Satish Narayanasamy, Cristiano Pereira, and Gilles Pokam. 2012. Maple: a coverage-driven testing tool for multithreaded programs. In *Proceedings of the 27th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications, OOPSLA 2012, part of SPLASH 2012, Tucson, AZ, USA, October 21-25, 2012*, Gary T. Leavens and Matthew B. Dwyer (Eds.). ACM, 485–502. doi:10.1145/2384616.2384651
- [72] Yuan Yu, Panagiotis Manolios, and Leslie Lamport. 1999. Model Checking TLA⁺ Specifications. In *Correct Hardware Design and Verification Methods, 10th IFIP WG 10.5 Advanced Research Working Conference, CHARME '99, Bad Herrenalb, Germany, September 27-29, 1999, Proceedings (Lecture Notes in Computer Science, Vol. 1703)*, Laurence Pierre and Thomas Kropf (Eds.). Springer, 54–66. doi:10.1007/3-540-48153-2_6
- [73] Xinhao Yuan and Junfeng Yang. 2020. Effective Concurrency Testing for Distributed Systems. In *ASPLOS '20: Architectural Support for Programming Languages and Operating Systems, Lausanne, Switzerland, March 16-20, 2020*, James R. Larus, Luis Ceze, and Karin Strauss (Eds.). ACM, 1141–1156. doi:10.1145/3373376.3378484
- [74] Michal Zalewski. [n. d.]. American fuzzy lop. <http://lcamtuf.coredump.cx/afl/>.
- [75] Andreas Zeller, Rahul Gopinath, Marcel Böhme, Gordon Fraser, and Christian Holler. 2019. The fuzzing book.
- [76] Yuqi Zhang, Yu Huang, Hengfeng Wei, and Xiaoxing Ma. 2024. Model-checking-driven explorative testing of CRDT designs and implementations. *J. Softw. Evol. Process.* 36, 4 (2024). doi:10.1002/SMR.2555

Received 2024-10-16; accepted 2025-08-12